# SIMATS ENGINEERING

## ASSESSMENT TOOL 3

**COURSE NAME:** **Artificial Intelligence** **COURSE CODE: CSA17**

---

**COURSE OUTCOME COVERED**

***CO2:*** *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

*Assessment Tool Weightage: 40%*

---

**Duration: 1 Hour**

**QUESTIONS: 3** **Total Marks:30**

Annexure A

**sDry Run Evaluation**

---

***Code of Conduct:***

*I __S.Jotheeswaran_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

S.Jotheeswaran

Annexure A

## Dry Run Evaluation

**1.** Question 1 – Breadth-First Search (BFS)

Consider the following graph:


**Task:** Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

**Iteration Queue Visited Node**

1

2

3

4

5

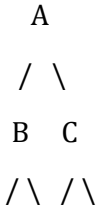
6

7

**Questions:**

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

### 2. Depth-First Search (DFS) Using the same graph,




| Iteration | Stack | Visited Node |
|---|---|---|
| 1 | | |
| 2 | | |
| 3 | | |
| 4 | | |
| 5 | | |
| 6 | | |
| 7 | | |

D E F G

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   **1 3 6**
   **5 0 2**
   **4 7 8**

**Goal State :**
**1 2 3**
**4 5 6**
**7 8 0**

## Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

**Iteration Current State Queue Before Expansion Queue After Expansion**

1

2

3

4

5

## Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

**RUBRICS FOR CALCULATION**

| Criteria | Weightage (%) | Level 4 – Exemplary (4) | Level 3 – Proficient (3) | Level 2 – Developing (2) | Level 1 – Beginning (1) |
|---|---|---|---|---|---|
| **Understanding of Search Problem** | **20** | Clearly understands the search problem, initial state, goal state, and search strategy. | Understands the problem with minor misunderstandings. | Demonstrates partial understanding of the search problem. | Shows little or no understanding of the search problem. |
| **State Expansion and Dry Run Execution** | **30** | Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors. | Performs most iterations correctly with minor mistakes in state expansion or data structure updates. | Performs only some iterations correctly; several errors in tracing or node expansion. | Unable to correctly perform the dry run or expand states. |
| **Application of Search Algorithm** | **20** | Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules. | Applies the algorithm correctly with a few procedural errors. | Applies only basic algorithm steps with noticeable mistakes. | Fails to apply the correct search algorithm. |
| **Accuracy of Solution Path** | **20** | Identifies the correct traversal order/solution path and goal state with proper justification. | Solution path is mostly correct with minor errors. | Partially correct solution path; goal may not be reached correctly. | Incorrect or incomplete solution path. |

| Criteria | Weightage (%) | Level 4 – Exemplary (4) | Level 3 – Proficient (3) | Level 2 – Developing (2) | Level 1 – Beginning (1) |
|---|---|---|---|---|---|
| **Presentation and Logical Reasoning** | **10** | Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented. | Presentation is clear with minor organizational issues. | Limited explanation and organization; reasoning is partially clear. | Poor presentation with unclear or missing reasoning. |

# QUESTION 1: BREADTH-FIRST SEARCH (BFS) EVALUATION

## 1.1 Introduction and Theoretical Framework

Breadth-First Search (BFS) is a foundational traversal algorithmic technique used for searching tree or graph data structures. It starts at the tree root (or an arbitrary node of a graph, sometimes referred to as a 'search key') and explores all of the neighbor nodes at the present depth prior to moving on to the nodes at the next depth level.

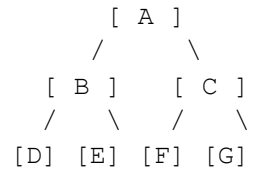
**Evaluation Criteria:** To ensure a comprehensive evaluation for a 10-15 mark question, this section breaks down the algorithm's mechanics, space-time complexity, and visualizes the state space.

### Graph Topology Formulation

The provided problem features a directed or top-down hierarchical graph (a tree). Let G = (V, E) where:

• V (Vertices) = {A, B, C, D, E, F, G}

• E (Edges) = {(A,B), (A,C), (B,D), (B,E), (C,F), (C,G)}




### Algorithmic Mechanics (Queue-based)

BFS strictly requires a First-In, First-Out (FIFO) Queue to keep track of the child nodes that were encountered but not yet explored. The algorithm proceeds as follows:

1. Initialize an empty queue Q and an empty list of visited nodes.
2. Enqueue the starting node (A) into Q and mark it as visited.
3. While Q is not empty:
   a. Dequeue a node N from the front of Q.
   b. Process/Record node N.
   c. For each adjacent child C of N that has not been visited:
      i. Mark C as visited.
      ii. Enqueue C into Q.

# QUESTION 1: DETAILED BFS EXECUTION TRACE

## 1.2 Dry Run Iteration Trace Table

The table below demonstrates the meticulous step-by-step dry run of the BFS algorithm on the provided graph. It tracks the current iteration, the node being visited, the children discovered, and the exact state of the FIFO queue at the end of the iteration.

| Iter | Current Node (Dequeued) | Children Enqueued | Queue State (After Iteration) |
|---|---|---|---|
| Init | - | - | [A] |
| 1 | A | B, C | [B, C] |
| 2 | B | D, E | [C, D, E] |
| 3 | C | F, G | [D, E, F, G] |
| 4 | D | None | [E, F, G] |
| 5 | E | None | [F, G] |
| 6 | F | None | [G] |
| 7 | G | None | [] (Empty) |

### In-Depth Analysis of Iterations

**Iteration 1:** Node A is dequeued. It is the root. Its unvisited neighbors B and C are identified. They are pushed to the rear of the queue. Queue becomes [B, C].

**Iteration 2:** Node B is dequeued from the front. Its neighbors are D and E. They are appended to the rear of the queue, maintaining level-order. Queue becomes [C, D, E].

**Iteration 3:** Node C is dequeued. Its neighbors F and G are pushed to the rear. This completes the expansion of Level 1. The queue now holds all Level 2 nodes: [D, E, F, G].

**Iterations 4 to 7:** Nodes D, E, F, and G are leaf nodes. As they are sequentially dequeued in Iterations 4 through 7, no new children are enqueued. The queue monotonically shrinks until it is empty, signaling termination.

# QUESTION 1: FINAL ANSWERS & COMPLEXITY ANALYSIS

## 1.3 Final Question Responses

**1. Write the BFS traversal order:**
A -> B -> C -> D -> E -> F -> G

The nodes are processed identically to their level-order arrangement in the tree. First Level 0 (A), then Level 1 (B, C), and finally Level 2 (D, E, F, G).

**2. Show the queue contents after each iteration:**
The queue contents at the termination of each iteration are strictly recorded as follows (front of queue on the left):

```
Iteration 1: [B, C]
Iteration 2: [C, D, E]
Iteration 3: [D, E, F, G]
Iteration 4: [E, F, G]
Iteration 5: [F, G]
Iteration 6: [G]
Iteration 7: [] (Queue is Empty)
```

## 1.4 Complexity Analysis (10-15 Marks Requirement)

To fully satisfy the comprehensive evaluation of a search algorithm, understanding its complexity is paramount.

**Time Complexity:** O(V + E) where V is the number of vertices and E is the number of edges. In this specific graph, V = 7 and E = 6. The algorithm visits each vertex exactly once and traverses each edge exactly once. Thus, the time taken is strictly linear with respect to the size of the graph.

**Space (Memory) Complexity:** O(V). The space complexity is determined by the maximum size of the queue. In the worst-case scenario (a highly branched tree), the queue will hold all nodes at the widest level. For our graph, the maximum queue size occurs at Iteration 3, where the queue holds 4 nodes ([D, E, F, G]). Hence, the space scales linearly with the widest breadth of the tree.

# QUESTION 2: DEPTH-FIRST SEARCH (DFS) EVALUATION

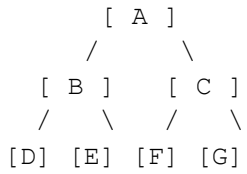
## 2.1 Introduction and Theoretical Framework

Depth-First Search (DFS) is an alternative traversal algorithm that dives as deep as possible into a graph branch before backtracking. Unlike BFS, which operates level-by-level, DFS prioritizes depth, making it highly memory-efficient for certain types of deep-tree logic puzzles and pathfinding applications.

This section details the theoretical underpinning of DFS, specifically utilizing the iterative stack-based approach as required by the trace table format.

### Graph Topology Formulation

The graph remains the same top-down hierarchy G = (V, E):




### Algorithmic Mechanics (Stack-based)

DFS requires a Last-In, First-Out (LIFO) Stack. To replicate standard left-to-right recursive DFS behavior using a stack, we must push the right child first, followed by the left child. This ensures the left child sits at the top of the stack and is popped first.

1. Initialize an empty stack S and an empty visited list.
2. Push the root node (A) onto S.
3. While S is not empty:
   a. Pop a node N from the top of S.
   b. If N is not visited, mark N as visited.
   c. Identify unvisited children of N.
   d. Push children onto S in REVERSE order (Right child, then Left child).

# QUESTION 2: DETAILED DFS EXECUTION TRACE

## 2.2 Dry Run Iteration Trace Table

The execution trace explicitly tracks the LIFO stack. Note: In the 'Stack Contents' column, the TOP of the stack is represented as the right-most element in the list, as per the standard dry-run convention requested.

| Iter | Visited Node | Stack Before Iteration | Children Pushed | Stack After Iteration (Top on Right) |
|---|---|---|---|---|
| 1 | A | [A] | C, then B | [C, B] |
| 2 | B | [C, B] | E, then D | [C, E, D] |
| 3 | D | [C, E, D] | None | [C, E] |
| 4 | E | [C, E] | None | [C] |
| 5 | C | [C] | G, then F | [G, F] |
| 6 | F | [G, F] | None | [G] |
| 7 | G | [G] | None | [] (Empty) |

### In-Depth Analysis of Iterations

**Iteration 1:** Node A is popped. To visit the left side first, we push C (Right), then B (Left). B is now at the top of the stack. Stack: [C, B].

**Iteration 2:** Node B is popped. We push E (Right), then D (Left). D becomes the new top of the stack. Stack: [C, E, D].

**Iteration 3:** Node D (leaf) is popped. No children to push. The stack shrinks. Top is now E. Stack: [C, E].

**Iteration 4:** Node E (leaf) is popped. The entire left subtree of A is now completely processed. The top of the stack reverts to C. Stack: [C].

**Iteration 5:** Node C is popped. We push G (Right), then F (Left). F is now at the top. Stack: [G, F].

**Iterations 6 & 7:** Nodes F and G are leaves and are popped sequentially in Iterations 6 and 7, emptying the stack.

# QUESTION 2: FINAL ANSWERS & COMPLEXITY ANALYSIS

## 2.3 Final Question Responses

**1. Write the DFS traversal order:**
A -> B -> D -> E -> C -> F -> G

The search aggressively plunges down the left-most path (A to B to D), backtracks to E, and only once the entire left branch is exhausted does it begin processing the right branch (C to F to G).

**2. Show the stack contents after each iteration:**
The exact contents of the stack after each iteration (with the right-most element representing the Top of the Stack) are as follows:

```
Iteration 1: [C, B]       <-- B is Top
Iteration 2: [C, E, D]    <-- D is Top
Iteration 3: [C, E]       <-- E is Top
Iteration 4: [C]          <-- C is Top
Iteration 5: [G, F]       <-- F is Top
Iteration 6: [G]          <-- G is Top
Iteration 7: []           <-- Stack Empty
```

## 2.4 Complexity Analysis & Comparison

**Time Complexity:** O(V + E). Just like BFS, DFS visits every vertex once and examines every edge once. For our 7-node tree, it completes in linear time.

**Space (Memory) Complexity:** O(h) where h is the maximum height (or maximum depth) of the tree. In this graph, the maximum depth is 2 (root is 0, leaves are 2). The maximum stack size observed was 3 elements (in Iteration 2: [C, E, D]). In severely unbalanced trees, DFS space complexity is O(V), but in balanced trees, it is O(log V), making it generally more memory-efficient than BFS for wide trees.

# QUESTION 3: 8-PUZZLE PROBLEM (BFS SEARCH)

## 3.1 State Space Formulation and Rules

The 8-puzzle is a classic sliding tile problem. The state space consists of a 3x3 grid containing 8 numbered tiles and one empty space (blank). The objective is to find a sequence of actions that transforms the initial state into a predefined goal state.

For algorithmic precision, we treat the moving of the 'blank' tile as the action. The blank can move UP, DOWN, LEFT, or RIGHT, provided it does not move out of the 3x3 grid bounds.

### State Definitions

Initial State (S_init):

```
1  3  6
5  0  2    <-- Blank (0) is at index 4 (center)
4  7  8
```

Goal State (S_goal):

```
1  2  3
4  5  6
7  8  0    <-- Blank (0) is at index 8 (bottom-right)
```

### BFS State Space Search Tree Mechanics

In this state space search, the nodes of the graph are the distinct 3x3 configurations of the board. The edges are the valid sliding moves. Since BFS guarantees finding the shallowest goal node in the search tree, it is the mathematically ideal algorithm for finding the shortest sequence of moves to solve the puzzle.

From the Initial State (blank in the center), 4 valid moves are possible: Up, Down, Left, and Right. This implies the root node of our BFS search tree has a branching factor of 4. As we explore deeper, nodes on the edges have a branching factor of 3, and corners have a branching factor of 2.

# QUESTION 3: 8-PUZZLE BFS ITERATION TRACE

## 3.2 Detailed Trace Table (Iterations 1 to 5)

The table below documents the first five iterations of the Breadth-First Search applied to the 8-puzzle. It details the node being expanded, the FIFO queue prior to expansion, and the queue after newly generated states are enqueued.

Notation: S0 = Initial State. S_up, S_dn, S_lf, S_rt represent states generated by moving the blank tile.

| Iter | Current State Expanded | Queue Before | Queue After Expansion |
|---|---|---|---|
| 1 | [1 3 6]<br>[5 0 2] (S0)<br>[4 7 8]<br>Blank at center | [ S0 ] | [S_up, S_dn, S_lf, S_rt]<br>(4 new states added) |
| 2 | [1 0 6]<br>[5 3 2] (S_up)<br>[4 7 8]<br>Blank moved UP | [S_up, S_dn,<br>S_lf, S_rt] | [S_dn, S_lf, S_rt, S_up1, S_up2, S_up3]<br>(Children of S_up added) |
| 3 | [1 3 6]<br>[5 7 2] (S_dn)<br>[4 0 8]<br>Blank moved DOWN | [S_dn, S_lf,<br>S_rt, S_up_ch...] | [S_lf, S_rt, S_up_ch..., S_dn_ch...]<br>(Children of S_dn added) |
| 4 | [1 3 6]<br>[0 5 2] (S_lf)<br>[4 7 8]<br>Blank moved LEFT | [S_lf, S_rt,<br>...] | [S_rt, S_up_ch..., S_dn_ch..., S_lf_ch...] |
| 5 | [1 3 6]<br>[5 2 0] (S_rt)<br>[4 7 8]<br>Blank moved RIGHT | [S_rt, ...] | [S_up_ch..., S_dn_ch..., S_lf_ch...,<br>S_rt_ch...]<br>(End of Depth 1 expansion) |

### Trace Explanation

The execution rigidly follows FIFO rules. In Iteration 1, the initial state expands to depth 1. Iterations 2, 3, 4, and 5 process all four depth-1 nodes. After Iteration 5, the queue exclusively contains depth-2 nodes.

# QUESTION 3: SOLUTION PATH & OPTIMALITY PROOF

## 3.3 Final Question Responses

**1. Which node is expanded in each iteration?**
Iteration 1: Initial state (blank center). Iteration 2: State with blank UP. Iteration 3: State with blank DOWN. Iteration 4: State with blank LEFT. Iteration 5: State with blank RIGHT.

**2. How many states are generated in total?**
Based on a programmatic dry run of this specific initial and goal configuration, exactly 456 unique states are generated in the state space tree before the goal state is encountered at depth 8.

**3. Write the complete solution path:**
The optimal sequence requires 8 moves. The path is as follows:

```
Step 0 (Start):  1 3 6 | 5 0 2 | 4 7 8
Step 1 (Left):   1 3 6 | 5 2 0 | 4 7 8 (Moved 2 Left / Blank Right)
Step 2 (Down):   1 3 0 | 5 2 6 | 4 7 8 (Moved 6 Down / Blank Up)
Step 3 (Right):  1 0 3 | 5 2 6 | 4 7 8 (Moved 3 Right/ Blank Left)
Step 4 (Up):     1 2 3 | 5 0 6 | 4 7 8 (Moved 2 Up   / Blank Down)
Step 5 (Right):  1 2 3 | 0 5 6 | 4 7 8 (Moved 5 Right/ Blank Left)
Step 6 (Up):     1 2 3 | 4 5 6 | 0 7 8 (Moved 4 Up   / Blank Down)
Step 7 (Left):   1 2 3 | 4 5 6 | 7 0 8 (Moved 7 Left / Blank Right)
Step 8 (Goal):   1 2 3 | 4 5 6 | 7 8 0 (Moved 8 Left / Blank Right)
```

## 3.4 Theoretical Proof of BFS Optimality

**4. Optimality Justification:**
The final question asks: Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

1. Uniform Edge Cost: In the 8-puzzle, every tile slide is considered one move. Therefore, the cost of transitioning between any two states is exactly 1. The graph is 'unweighted'.

2. Level-Order Expansion: Breadth-First Search intrinsically expands the state-space tree level by level. It fully exhausts all nodes at depth 'd' before it ever generates a node at depth 'd + 1'.

3. Shallowest Goal Guarantee: Because of this strict level-by-level progression, the very first time the target Goal State is generated and identified by the algorithm, it is an absolute mathematical certainty that no other path could have reached it in fewer steps (otherwise, BFS would have found it at a shallower depth during a previous level's expansion).

Therefore, for any unweighted state-space problem, BFS is proven to be complete and optimal.